

ASSIGNMENT – 10

NAME : M.AKSHAY PATEL

HALL TICKET NO : 2303A51745

BATCH : 11

Question :

Exploring Layer 2 – Testnet

Objective To understand and explore Layer 2 blockchain networks by deploying and interacting with a simple smart contract on a Layer 2 Testnet (e.g., Polygon Mumbai / Arbitrum Sepolia), and observe reduced gas fees and faster transactions compared to Layer

Requirements :

- Basic understanding of blockchain and smart contracts
- MetaMask wallet installed (browser extension)
- Testnet ETH/MATIC from faucet
- VS Code (with Solidity extension installed)
- Node.js and npm installed
- Hardhat development framework Tools & Technologies Used
- Blockchain Type: Layer 2 Testnet
- Network: Polygon Mumbai Testnet (or Arbitrum Sepolia)

Language: Solidity

Framework: Hardhat

Wallet: MetaMask

Execution Key is:- 0x742d35Cc6634C0532925a3b844Bc454e4438f44e

CODE :

```
import tkinter as tk from tkinter
import messagebox import
random import time class DeFi:
    def __init__(self):
self.balances = {}
def
gas_fee(self):
```

```

        return round(random.uniform(0.0001, 0.001),
6) def delay(self):    time.sleep(0.3) def deposit(self,
user, amount):        if amount <= 0:
    return "Invalid amount"

    self.delay()    gas = self.gas_fee()    if user not in self.balances:
self.balances[user] = 0    self.balances[user] += amount    return f"Deposit
Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"    def withdraw(self,
user, amount):        if user not in self.balances:        return "User not found"
if self.balances[user] < amount:        return "Insufficient balance"    self.delay()
gas = self.gas_fee()    self.balances[user] -= amount    return f"Withdraw
Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"

    def balance(self, user):
        return self.balances.get(user, 0)

class App:
    def __init__(self, root):
        self.defi = DeFi()    self.root = root
self.root.title("DeFi Dashboard")
self.root.geometry("600x400")    left_frame =
tk.Frame(root, bg="#2c3e50", width=200)
left_frame.pack(side="left", fill="y")    tk.Label(left_frame,
text="DeFi Panel", fg="white",        bg="#2c3e50",
font=("Arial", 14, "bold")).pack(pady=20)
tk.Label(left_frame, text="User", fg="white",
bg="#2c3e50").pack()    self.user_entry =
tk.Entry(left_frame)    self.user_entry.pack(pady=5)
tk.Label(left_frame, text="Amount", fg="white",
bg="#2c3e50").pack()    self.amount_entry =
tk.Entry(left_frame)    self.amount_entry.pack(pady=5)
tk.Button(left_frame, text="Deposit", width=15,
command=self.deposit).pack(pady=10)
tk.Button(left_frame, text="Withdraw", width=15,

```

```

command=self.withdraw).pack(pady=5)

tk.Button(left_frame, text="Check Balance", width=15,
command=self.check_balance).pack(pady=5)

right_frame = tk.Frame(root, bg="#ecf0f1")
right_frame.pack(side="right", expand=True, fill="both")

tk.Label(right_frame, text="Transaction Output",
bg="#ecf0f1", font=("Arial", 14, "bold")).pack(pady=10)

self.output = tk.Text(right_frame, height=15, width=50)
self.output.pack(padx=10, pady=10)

def deposit(self):
    user = self.user_entry.get()

    try:
        amount =
float(self.amount_entry.get())        result =
self.defi.deposit(user, amount)
self.show(result)    except:
        messagebox.showerror("Error", "Invalid input")

def withdraw(self):
    user = self.user_entry.get()
    try:
        amount = float(self.amount_entry.get())

    result = self.defi.withdraw(user, amount)

    self.show(result)    except:
        messagebox.showerror("Error", "Invalid input")

def check_balance(self):
    user = self.user_entry.get()    balance =
self.defi.balance(user)    self.show(f"{user}
Balance: {balance} ETH")

def show(self, text):
    self.output.delete(1.0, tk.END)
    self.output.insert(tk.END, text)

if __name__ == "__main__":

```

```
root = tk.Tk()
```

```
app = App(root)
```

```
root.mainloop()
```

IMPLEMENTATION :

```
C:\> Users > Rushi > Downloads > gui.py > DeFi
1  import tkinter as tk
2  from tkinter import messagebox
3  import random
4  import time
5  class DeFi:
6      def __init__(self):
7          self.balances = {}
8      def gas_fee(self):
9          return round(random.uniform(0.0001, 0.001), 6)
10     def delay(self):
11         time.sleep(0.3)
12     def deposit(self, user, amount):
13         if amount <= 0:
14             return "Invalid amount"
15         self.delay()
16         gas = self.gas_fee()
17         if user not in self.balances:
18             self.balances[user] = 0
19         self.balances[user] += amount
20         return f"Deposit Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"
21     def withdraw(self, user, amount):
22         if user not in self.balances:
23             return "User not found"
24         if self.balances[user] < amount:
25             return "Insufficient balance"
26         self.delay()
27         gas = self.gas_fee()
28         self.balances[user] -= amount
29         return f"Withdraw Success\nUser: {user}\nAmount: {amount} ETH\nGas: {gas}"
30     def balance(self, user):
31         return self.balances.get(user, 0)
32 class App:
33     def __init__(self, root):
```

```
C:\> Users > Rushi > Downloads > gui.py > DeFi
32 class App:
33     def __init__(self, root):
56         tk.Label(right_frame, text="Transaction Output",
57                 bg="#ecf0f1", font=("Arial", 14, "bold")).pack(pady=10)
58         self.output = tk.Text(right_frame, height=15, width=50)
59         self.output.pack(padx=10, pady=10)
60     def deposit(self):
61         user = self.user_entry.get()
62         try:
63             amount = float(self.amount_entry.get())
64             result = self.defi.deposit(user, amount)
65             self.show(result)
66         except:
67             messagebox.showerror("Error", "Invalid input")
68     def withdraw(self):
69         user = self.user_entry.get()
70         try:
71             amount = float(self.amount_entry.get())
72             result = self.defi.withdraw(user, amount)
73             self.show(result)
74         except:
75             messagebox.showerror("Error", "Invalid input")
76     def check_balance(self):
77         user = self.user_entry.get()
78         balance = self.defi.balance(user)
79         self.show(f"{user} Balance: {balance} ETH")
80     def show(self, text):
81         self.output.delete(1.0, tk.END)
82         self.output.insert(tk.END, text)
83 if __name__ == "__main__":
84     root = tk.Tk()
85     app = App(root)
86     root.mainloop()
```

OUTPUT :

